



# CMPT 295

Unit - Machine-Level Programming

Lecture 21 – Assembly language –  
Function Call and Stack - Managing Local Data –  
Recursion

gcc and us  
(software developers)  
must abide to ...

# Last Lecture

➤ Passing data mechanism - **x86-64 function call convention:** protocol

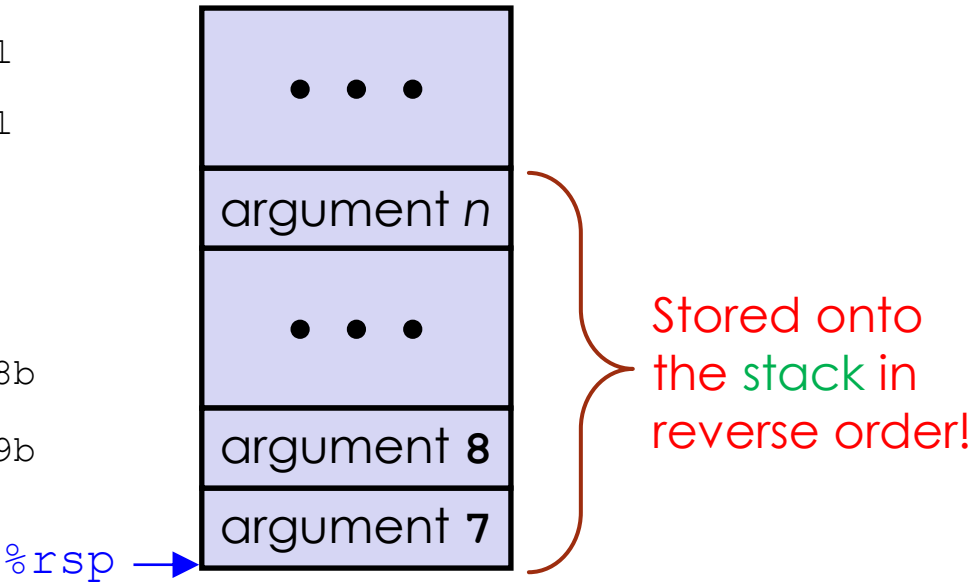
First 6 arguments

|               |             |                          |
|---------------|-------------|--------------------------|
| argument 1 -> | <b>%rdi</b> | or %edi, or %di or %dil  |
| argument 2 -> | <b>%rsi</b> | or %esi, or %si or %sil  |
| argument 3 -> | <b>%rdx</b> | or %edx, or %dx or %dl   |
| argument 4 -> | <b>%rcx</b> | or %ecx, or %cx or %cl   |
| argument 5 -> | <b>%r8</b>  | or %r8d, or %r8w or %r8b |
| argument 6 -> | <b>%r9</b>  | or %r9d, or %r9w or %r9b |

return value

**%rax**

Remaining arguments  
Stack



➤ If an arguments is a **pointer**, **caller** must make use of the **stack** in order to create such memory address before storing this memory address either in the appropriate register (if < 7<sup>th</sup> argument) or on the stack (if >= 7<sup>th</sup> argument)

# Today's Menu

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, move operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – **Stack** – Recursion
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point operations

# To recap ...

Overview of Function Call

What happens when a function (**caller**) calls another function (**callee**) in **x86-64 Assembly Language**?

1. **Control** (execution flow)
2. **Data**
3. **Memory** -> stack memory segment

Looking into the **function call conventions** of **x86-64 Assembly Language**

8

## 3. **Memory**

- **Memory** is allocated when **callee** function starts executing
- **Memory** is deallocated when **callee** function stops executing

} allocated  
a **stack**  
**frame on**  
the **stack**

What if a function needs storage space for its local data?

### 3. Managing local data

- When writing assembly programs, what can we use when we need storage space for our local data?

#### 1. We can use **registers**!

- Yes! Registers are our **first choice** as they are the fastest storage location on a computer
- OK! But, since we have observed that **registers are shared by all functions** in **x86-64** assembly language, we need to follow some convention, otherwise ... :

caller

```
who:
    . . .
    movq $295, %rbx
    call amI
    addq %rbx, %rax
    . . .
    ret
```

callee

```
amI:
    . . .
    subq $225, %rbx
    . . .
    ret
```

| Register Table |    |
|----------------|----|
| %rbx           |    |
| %rax           | 10 |

### 3. Managing local data - **register saving convention**: 1. **callee saved registers**

- When we need storage space for our local data ...

#### 1. Registers

- A function **needing space for local data** can use registers

- Some registers are referred to as **callee saved registers**:

  - **%rbx, %rbp, %r12 to %r15** (and **%ebx, %bx, %b1, ...**)

- **Callee saved registers** means that ...

  - ... the **callee** function must preserve the values of these registers before using them,

  - ... then **callee** function must restore their values before the control is returned to the **caller** function (through the execution of **ret** instruction)

**Register saving convention:**

1) **callee saved registers**

=> **callee's responsibility** to save/restore **callee saved registers**

## 3. Managing local data - *register saving convention*: 1. *callee saved registers*

- How can **callee** preserve the values of these **callee saved registers** before using them?
- Example of a scenario:
  - **Caller** needs space for local data, therefore **caller** chooses to use **%r13**
  - **Caller** calls **callee**
  - **Callee** needs space for local data, therefore **callee** chooses to use **%r13** which means that at the start of **callee**, **callee** must **pushq %r13** to preserve its value
  - Then **callee** uses **%r13**
  - Then before control (execution flow) returns from **callee** to **caller** (via **ret**), **callee** must **popq %r13**
  - The execution flow returns to **caller** which continues using **%r13** which now contains the **caller**'s expected value

### Result of register saving convention

1. **callee saved registers** says: upon return from **callee**, **caller** can always assume that these registers still contain the values **caller** stored in them before calling **callee**!

If **callee** pushes more than 1 register, then **callee** pops them in reverse order

## 3. Managing local data - **register saving convention**: 2. **caller saved registers**

### 1. Registers (cont'd)

**register saving convention:**

2) **caller saved registers**

=> **caller's responsibility** to save/restore **caller saved registers**

- Some registers are referred to as **caller saved registers**:
  - **%r10, %r11, %rax** and all the 6 registers used for passing data as arguments to **callee** (and **%r10d, %r10w, %r10b, ...**)
- **Caller saved registers** means that ...
  - the **caller** function must preserve the values of these registers before ...
    - ... setting up the **callee's** argument(s) into the appropriate "data passing as argument" register(s) and
    - ... calling the **callee**
  - then once the control is returned to the **caller**, the **caller** must restore their values before using them



## 3. Managing local data - **register saving convention**: 2. **caller saved registers**

- How can **caller** preserve the values of these **caller saved registers** before setting up the **callee**'s argument(s) and calling the **callee**?

- Example of a scenario:

- Caller** needs space for local data, therefore **caller** chooses to use **%r10**
- The implication of this choice is that before calling **callee**, **caller** must **pushq %r10** to preserve its value then calls **callee**
- Callee** needs space for local data, therefore **callee** chooses to use **%r10**
- Then after the execution flow has returned from **callee** to **caller** (via **ret**), **caller popq %r10** to restore its value
- Caller** continues using **%r10** which now contains the expected value

### Result of register saving convention

2. **caller saved registers** says: **callee** can always assume that **caller** has saved the content of these registers, so it is **safe** for **callee** to use them!

If **caller** pushes more than 1 register, then **caller** pops them in reverse order

# x86-64 register saving convention

%rsp →

M[]  
Stack

## ➤ Solution 1:

```
who:
    . . .
    movq $295, %rbx
    call amI
    addq %rbx, %rax
    . . .
    ret
```

```
amI:
    . . .
    subq $225, %rbx
    . . .
    ret
```

Register Table:

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |

## ➤ Solution 2:

```
who:
    . . .
    movq $295, %r10
    . . .

    call amI

    . . .
    addq %r10, %rax
    . . .
    ret
```

```
amI:
    . . .
    subq $225, %r10
    . . .
    ret
```

### 3. Managing local data => **spilling**

- When writing assembly programs, what can we use when we need storage space for our local data?

- **We can use stack!**

If we run out of registers!

#### 2. Stack

- A function can use the **stack** to store the values of its local variables and for temporary space
- Example of **Set-up** code (creating space on the **stack**):

- **subq \$16, %rsp** OR **pushq %rax**

- Example of code that **spills** onto the **stack**:

- **movq %rax, 8(%rsp)**

- Example of **Clean-up** code (restoring the **stack**):

- **addq \$16, %rsp** OR **popq %rax**

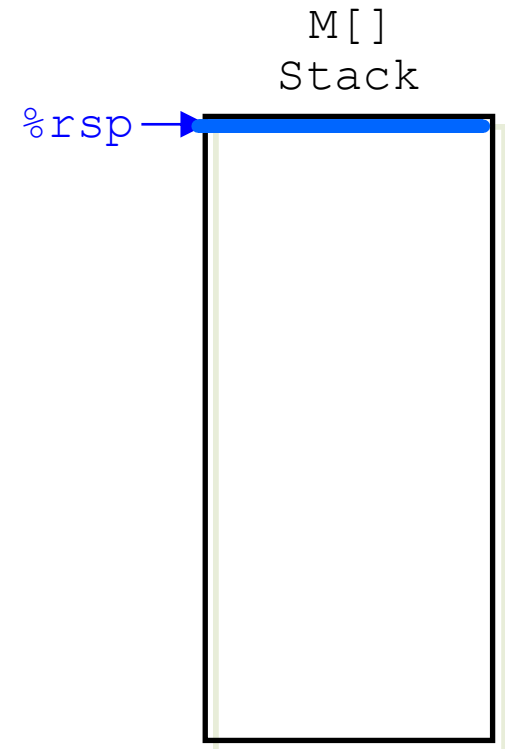
**Callee** must restore the **stack** before control is returned to **caller**!

# Local variables on Stack – Example

```
long call_incr() {  
    long v1 = 295;  
    long v2 = incr(&v1, 225);  
    return v1+v2;  
}
```

```
long incr(long *p, long val)  
{  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $295, 8(%rsp)  
    movl    $225, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```



$$\frac{\text{base} + \text{displacement}}{2}$$

## Stack Segment

### Purpose

Here is the opportunity to get some practice drawing the result of hand tracing the code on the previous slide!

### Register Table :

[illegible][illegible]

M [ ]

Stack

Bottom

Stack frame  
of 1<sup>st</sup>  
invocation  
of *func*

Stack frame  
of 2<sup>nd</sup>  
invocation  
of *func*

Stack frame  
of 3<sup>rd</sup>  
invocation  
of *func*

Top

# Recursion in x86-64

We already know  
how to implement  
**recursive functions**  
in x86-64 assembly  
language!

- Handled without special instruction

1. Call / return mechanism still follows the *stack pattern*

➤ If *func* (first invocation of *func*) calls itself (second invocation of *func*), then the second invocation of *func* must terminate before the first invocation of *func* can do so

2. Each invocation of the recursive function has its own *stack frame*

- Saved registers & local variables

- Argument 7.. n (if any)

- Saved return address

- *Function call conventions* still required and implemented

- Passing *arguments* and returning *return value*

- *Register saving conventions* still required and implemented

- Prevent one function invocation from corrupting another's data (stored in registers)

- Also works for mutual (or indirect) recursion

- *func1* calls *func2* then *func2* calls *func1*

# Recursive Function – countOnesR ( ... )

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

**What does this function do?**

# Recursive Function – Example – Base Case

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

countOnesR:

```
xorl    %eax, %eax  
testq   %rdi, %rdi  
je       done  
pushq   %rbx  
movq    %rdi, %rbx  
andl    $1, %ebx  
shrq    %rdi  
call    countOnesR  
addq    %rbx, %rax  
popq    %rbx
```

```
done:  
ret
```



# Recursive Function – Example – Callee saved registers

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

# Recursive Function – Example – Arguments setup

```
/* Recursive counter of 1's */
long countOnesR(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1) + countOnesR(x >> 1);
}
```

```
countOnesR:
    xorl    %eax, %eax
    testq   %rdi, %rdi
    je      done
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    countOnesR
    addq    %rbx, %rax
    popq    %rbx
done:
    ret
```

# Recursive Function – Example – Recursive Call

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

# Recursive Function – Example – Result

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

# Recursive Function – Example – Restore stack and return

```
/* Recursive counter of 1's */  
long countOnesR(unsigned long x) {  
    if (x == 0)  
        return 0;  
    else  
        return (x & 1) + countOnesR(x >> 1);  
}
```

```
countOnesR:  
    xorl    %eax, %eax  
    testq   %rdi, %rdi  
    je      done  
    pushq   %rbx  
    movq    %rdi, %rbx  
    andl    $1, %ebx  
    shrq    %rdi  
    call    countOnesR  
    addq    %rbx, %rax  
    popq    %rbx  
done:  
    ret
```

# Recursive Function – Example – Test Cases

- ▶ Test Case 1
  - ▶ Input:  $x = 0$
  - ▶ Expected result: 0
- ▶ Test Case 2
  - ▶ Input:  $x = 7$
  - ▶ Expected result: 3

$$\frac{\text{base} + \text{displacement}}{2}$$

## Stack Variables

For you to practice  
drawing the stack

### Register Table :

[illegible][illegible]

## countOnesR:

```
xorl    %eax, %eax
```

```
testq    %rdi, %rdi
```

je                      done

**pushq     %rbx**

```
movq    %rdi, %rbx
```

```
andl    $1, %ebx
```

**shrq      %rdi**

**call**      **countOnesR**

```
addq    %rbx, %rax
```

popq      %rbx

**done :**

ret

# Summary

- **x86-64 register saving convention**

## Responsibility of **callee**

### 1. **callee** saved registers:

- **Callee** must save before using register & must restore register before control returns to **caller**

|              |
|--------------|
| % <u>rbx</u> |
| %r12         |
| %r13         |
| %r14         |
| %r15         |
| % <u>rbp</u> |
| % <u>rsp</u> |

## Responsibility of **caller**

### 2. **caller** saved registers:

- **Caller** must save register before calling **callee** & must restore register after control has returned to **caller**

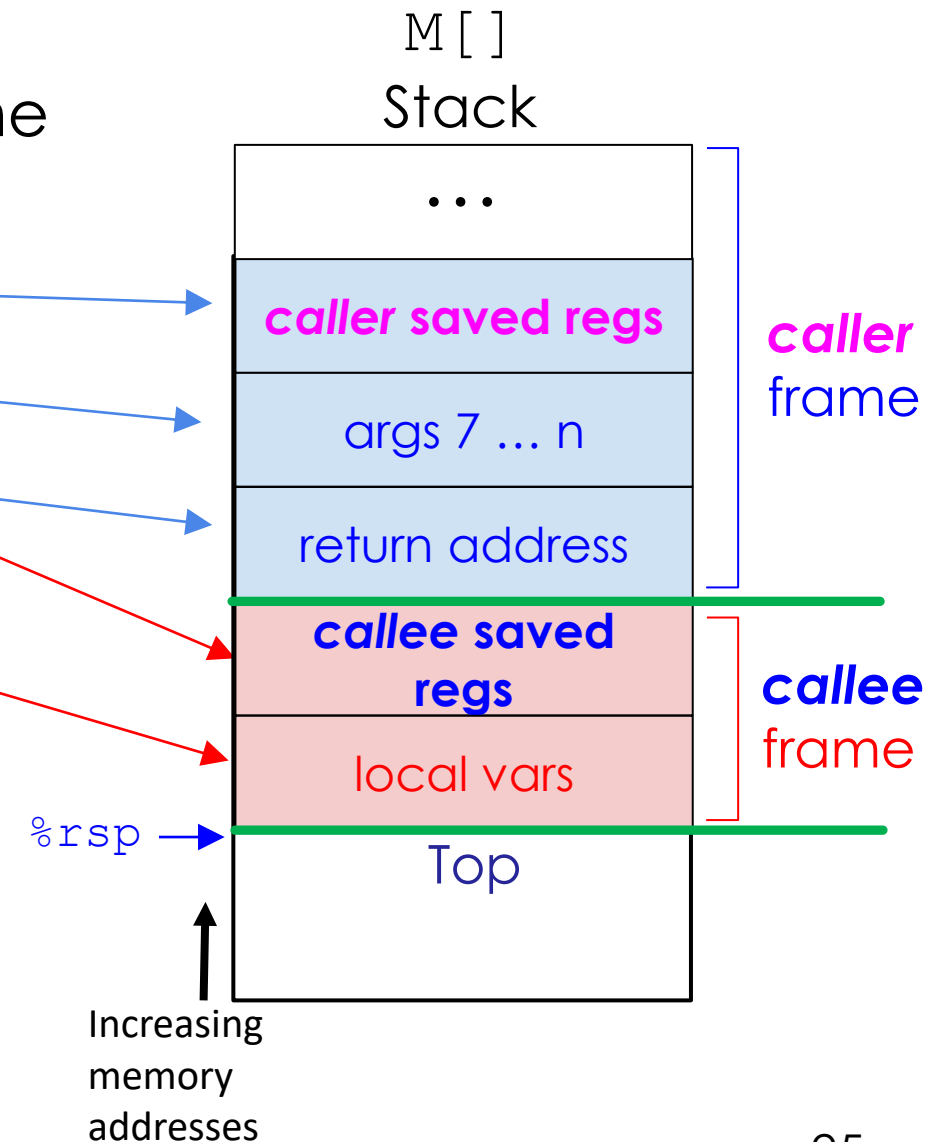
|                          |              |
|--------------------------|--------------|
| Return value             | % <u>rax</u> |
| Parameters/<br>arguments | % <u>rdi</u> |
|                          | % <u>rsi</u> |
|                          | % <u>rdx</u> |
|                          | % <u>rcx</u> |
|                          | %r8          |
|                          | %r9          |
|                          | %r10         |
|                          | %r11         |



# Summary

- x86-64 conventions and stack frame

**caller** preserves **caller saved registers**  
**caller** passes arguments  
**caller** calls **callee**  
**callee** preserves **callee saved registers**  
**callee** constructs local vars (get stack space)  
**callee** performs function  
**callee** restores stack space (recycles local vars)  
**callee** restores **callee saved registers**  
**callee** returns to **caller**  
**caller** restores stack space (recycles arguments)  
**caller** restores **caller saved registers**



# Summary

- Recursion

- Handled without special instruction

- Using stack frames

- Using x86-64 function call and Register saving conventions

# Next lecture

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point operations